UFOCatch □□□□□□□□□

□□□□□□□□

DirectX & C++ □□□□□

□□□□□□□□

□□□□□□□□: UFO□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

□□□□:
- □□□□: C++
- □□□□□□□API: DirectX 11
- □□□□□□□: □□□□□□□□□□□□□□□□□
- □□□□□□□□: State Pattern, Object Pool, Singleton

□□□□: □□□□
□□□□: □10,000□□□□□□□□□□□

4□□□□□□□□

1. □□□□□□□□□□□□□□□4□□ + □□□□□□□

□□□□: Src/09_Enemy/Liner4Tree.h

□□□□:
- 4□□□□□□□□□□□□□□□□□□□
- □□□□□□□□Z□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□ O(n²) □□ O(n log n) □□□

□□□□□□□□□:
- □□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

```
// Morton number calculation with bit separation
uint16_t BitSeparate(uint16_t n) {
    n = (n | (n << 8)) & 0x00ff00ff;
    n = (n | (n << 4)) & 0x0f0f0f0f;
    n = (n | (n << 2)) & 0x33333333;
    n = (n | (n << 1)) & 0x55555555;
    return n;
}

uint16_t Get2DMortonNumber(float worldX, float worldY) {
    uint16_t cellX = normalizeCoord(worldX);
    uint16_t cellY = normalizeCoord(worldY);
    return BitSeparate(cellX) | (BitSeparate(cellY) << 1);
}
```

2. □□□□□□□□□□□□□□

□□□□: Src/11_GameSystem/VisionSystem.h/cpp

□□□□:
- AI□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- 2□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □AI□□□□□□□□□□□□□□□

□□□□□□□□□:
- □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

```
  // Line segment-circle intersection detection
  bool LineSegmentCircleIntersection(
      const VECTOR2& lineStart, const VECTOR2& lineEnd,
      const VECTOR2& circleCenter, float circleRadius) const {

      VECTOR2 direction = lineEnd - lineStart;
      VECTOR2 toCenter = lineStart - circleCenter;

      float a = dot(direction, direction);
      float b = 2.0f * dot(toCenter, direction);
      float c = dot(toCenter, toCenter) - radius * radius;

      float discriminant = b * b - 4.0f * a * c;
      if (discriminant < 0.0f) return false;

      float t1 = (-b - sqrt(discriminant)) / (2.0f * a);
      float t2 = (-b + sqrt(discriminant)) / (2.0f * a);

      return (t1 >= 0 && t1 <= 1) || (t2 >= 0 && t2 <= 1);
  }
```

3. □□□□□□□□□□□□□□State Pattern□

□□□□□: Src/12_Tutorial/TutorialState.h, Src/09_Enemy/Human/State/

□□□□□:
- □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□: Move → Suction → Expand → Discovery → Play
- □AI□□□□□: Idle → Walk → FindPlayer

□□□□□□□□□:
- □□□□□□: Actor□□□□State□□□□□□□
- std::unordered_map □□□ O(1) □□□□□□
- □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□□□

```
// State pattern implementation
class CTutorialState {
public:
    virtual void Enter() {}
    virtual void Update() {}
    virtual void Exit() {}

    enum class State {
        Move, Suction, Expands, Discovery, Play
    };
};

class CMoveState : public CTutorialState {
    void Update() override {
        // Handle movement tutorial logic
        if (playerMoved()) transitionToNext();
    }
};

// State management with unordered_map
std::unordered_map<StateType, std::unique_ptr<State>> states;
```

4. □□□□□□□□□□□□□□□□□□□□

□□□□□: Src/07_Scene/, Src/08_Player/, Src/11_GameSystem/

□□□□□:
- □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□
- □□□□□□□□□□: □□□□□□□□□□□□□□□□□□Lerp□□

□□□□□□□□□□:
- DeltaTime□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□

```
// Suction displacement calculation
VECTOR3 CalcSuctionDisplacement(float moveTime,
                                const VECTOR3& animalPos) const {
    // Calculate speed multiplier based on height
    float heightDiff = m_coneTopPos - animalPos.y;
    float progress = 1.0f - (heightDiff / m_coneTopPos);
    float eased = progress * progress * progress; // Cubic easing

    float speedMultiplier = lerp(minSpeed, maxSpeed, eased);

    // Project to ground plane using internal division
    float ratio = (0 - animalPos.y) / (animalPos.y - m_coneTopPos);
    VECTOR3 groundPoint = VECTOR3(
        animalPos.x + ratio * (animalPos.x - transform.position.x),
        0,
        animalPos.z + ratio * (animalPos.z - transform.position.z)
    );

    VECTOR3 pullVector = transform.position - groundPoint;
    return pullVector / moveTime * speedMultiplier * DeltaTime();
}
```

[illegible]

[illegible]:
- [illegible]lizard[illegible]
- [illegible]
- [illegible]

[illegible]:
- [illegible]90%[illegible]
- [illegible]
- [illegible]

[illegible]:
- [illegible]
- [illegible]DRY[illegible]
- [illegible]SOLID[illegible]

[illegible]:
[illegible]:
• [illegible]C++[illegible]STL[illegible]C++11/14[illegible]
• [illegible]
• [illegible]DirectX[illegible]
• [illegible]
• [illegible]
• [illegible]